DevOps Technical Interview

Question & Answer Reference Guide

Kubernetes · AWS · Terraform · Docker · Linux · CI/CD & Git

#Kubernetes	#AWS	#Terraform	#Docker	#Linux	#CI/CD	#Git

Real questions from a DevOps technical interview — structured for study and revision.

[#K8s] Kubernetes

Q: 1. How do you separate environments (Dev, Staging, Prod) in Kubernetes or across clusters?

Two main strategies:

- **Namespace-based (single cluster):** Create separate namespaces — dev, staging, prod. Use RBAC to restrict access per namespace. Apply ResourceQuotas and LimitRanges per namespace. Use NetworkPolicies to isolate traffic.
- **Cluster-per-environment (multi-cluster):** Dedicated clusters for each environment. Stronger blast-radius isolation. Recommended for production workloads with strict compliance needs.
- **Tooling:** Helm values files per environment (values-dev.yaml, values-prod.yaml). Kustomize overlays (base + overlays/dev, overlays/prod). GitOps with Argo CD — separate Applications or ApplicationSets targeting different clusters/namespaces.

Note: Namespace isolation is logical, not physical. For hard security boundaries (PCI, HIPAA), separate clusters are preferred.

Q: 2. What is the purpose of a ServicePort in Kubernetes?

A **ServicePort** defines how a Kubernetes Service exposes a port to route traffic to Pods. It has three key fields:

- **port**: The port the Service listens on (what clients connect to).
- **targetPort**: The port on the Pod/container where traffic is forwarded.
- **nodePort**: (NodePort/LoadBalancer only) Exposes the service on each node's IP at a static port.

```
port: 80 # Service port (external)
targetPort: 8080 # Container port (internal)
protocol: TCP
```

This decoupling lets you change the container's port without altering how clients connect to the Service.

Q: 3. What is a Deployment in Kubernetes, and how does it manage Pods?

A **Deployment** is a higher-level abstraction that manages a ReplicaSet, which in turn manages Pods. It provides:

- **Desired state management:** Ensures the specified number of Pod replicas are always running.

- **Rolling updates:** Updates Pods incrementally (controlled by maxSurge and maxUnavailable) with zero downtime.
- **Rollback:** kubectl rollout undo deployment/<name> reverts to the previous ReplicaSet.
- **Self-healing:** Replaces failed/deleted Pods automatically.
- **Scaling:** kubectl scale deployment/<name> --replicas=5

Note: Deployments are stateless workloads. For stateful apps (DBs, Kafka), use StatefulSets.

[#AWS] AWS (Amazon Web Services)

Q: 1. What are the primary differences between AWS EC2 and AWS Fargate?

Dimension	EC2	Fargate
Server Management	You manage OS, patching, capacity	Fully serverless — AWS manages infra
Pricing	Pay per instance (hourly/reserved)	Pay per vCPU & memory used by task
Scaling	Manual/ASG — instance-level	Task-level, automatic
Use Case	Long-running, customised workloads	Event-driven, microservices, batch
Control	Full OS-level access	No access to underlying host

Q: 2. Explain the different Routing Policies available in Route 53.

- **Simple:** Single resource; no health checks. Default policy.
- **Weighted:** Split traffic by percentage across multiple resources. Used for A/B testing or gradual migrations.
- **Latency-Based:** Routes to the AWS region with the lowest latency for the user.
- **Failover:** Active-passive setup. Routes to secondary if primary health check fails.
- **Geolocation:** Routes based on the user's geographic location (continent/country).
- **Geoproximity:** Routes based on location + optional bias to shift more/less traffic. Requires Traffic Flow.
- **Multi-Value Answer:** Returns up to 8 healthy records. Not a load balancer, but improves availability.
- **IP-Based:** Routes based on the client's IP CIDR range. Useful for on-premises routing.

Q: 3. What is the use of a Rotation Policy in AWS KMS?

AWS KMS **automatic key rotation** periodically creates new cryptographic key material for a KMS key while keeping the same Key ID and ARN. Key points:

- Rotation frequency: Every 365 days by default (configurable 90–2560 days for customer-managed keys).
- Old key material is retained — data encrypted with old versions can still be decrypted.
- New data encrypted after rotation uses the latest key material.
- Purpose: Reduces the risk if key material is ever compromised; meets compliance requirements (SOC2, PCI-DSS).

Note: Rotation does NOT re-encrypt existing data. It only ensures future encryptions use fresh material.

Q: 4. What are the main parameters/attributes used when configuring Auto Scaling on AWS?

- **Launch Template / Launch Configuration:** Defines AMI, instance type, key pair, security groups, user data.
- **Min / Max / Desired Capacity:** Floor, ceiling, and starting count of instances.
- **Scaling Policies:** Target Tracking (e.g., keep CPU at 50%), Step Scaling, Simple Scaling, Scheduled Scaling.
- **Health Check Type:** EC2 health checks or ELB health checks.
- **Cooldown Period:** Time (seconds) after a scaling activity before another can start. Prevents thrashing.
- **Warm-up Period:** For target tracking — time for new instances to initialise before metrics are counted.
- **Termination Policy:** Which instance to terminate first (OldestInstance, NewestInstance, ClosestToNextHour, etc.).
- **VPC Subnets / AZs:** Distributes instances across AZs for high availability.

[#IaC] Terraform (Infrastructure as Code)

Q: 1. What is the purpose of the depends_on meta-argument in Terraform?

depends_on creates an explicit dependency between resources when Terraform cannot infer it automatically from attribute references.

- Terraform normally builds a dependency graph from resource references. depends_on is needed for hidden/implicit dependencies (e.g., an IAM policy must be attached before a Lambda function that uses it, but the Lambda resource doesn't directly reference the policy resource).

```
resource "aws_lambda_function" "example" {
...
depends_on = [aws_iam_role_policy_attachment.lambda_policy]
}
```

Note: Overuse of depends_on can slow plans and introduce unnecessary sequencing. Use only when the implicit graph is insufficient.

Q: 2. How do you view or extract outputs in Terraform after a deployment?

- **terraform output** — lists all outputs defined in the configuration after apply.
- **terraform output <name>** — retrieves a specific output value.
- **terraform output -json** — outputs all values as JSON (useful for scripting).
- **terraform output -raw <name>** — prints raw string value without quotes (useful in shell scripts).

```
# In outputs.tf
output "cluster_endpoint" {
value = aws_eks_cluster.main.endpoint
}

# CLI
terraform output cluster_endpoint
```

Note: Outputs are also readable from remote state using terraform_remote_state data source in other configurations.

Q: 3. What is the use of terraform taint, and when should it be applied?

terraform taint marks a resource for destruction and recreation on the next terraform apply — even if no configuration has changed.

- **When to use:** Resource is in a degraded or broken state that Terraform doesn't detect. Manual changes were made outside Terraform (config drift). A resource needs to be force-replaced (e.g., stale EC2 instance, corrupt EBS volume).

```
terraform taint aws_instance.web # Deprecated syntax
terraform apply -replace=aws_instance.web # Preferred (v0.15.2+)
```

Note: terraform taint is deprecated in Terraform v0.15.2+. Use terraform apply -replace=<resource> instead.

[#Docker] Docker

Q: 1. What is Docker Compose, and why is it used?

Docker Compose is a tool for defining and running multi-container Docker applications using a YAML file (docker-compose.yml).

- **Why use it:** Eliminates running multiple docker run commands manually. Defines services, networks, volumes, and environment variables in one file. Simplifies local development and testing of microservices. Single command to bring up (docker-compose up) or tear down (docker-compose down) the entire stack.

Note: For production, Docker Swarm or Kubernetes is preferred. Compose is primarily a developer/testing tool.

Q: 2. How can you run multiple containers on the same host port in Docker?

You **cannot** bind two containers to the exact same host port simultaneously — the OS only allows one process per port. Solutions:

- **Reverse Proxy / Load Balancer:** Use Nginx or HAProxy on port 80/443 that proxies to containers on different internal ports (e.g., 8081, 8082). This is the standard pattern.
- **Docker Swarm / Kubernetes:** The orchestrator handles port routing internally via its networking layer (kube-proxy, ingress controllers).
- **Different host ports:** Map each container to a unique host port (8081:8080, 8082:8080) and put a load balancer in front.

Q: 3. What is the purpose of the EXPOSE command in a Dockerfile?

EXPOSE is a *documentation instruction*. It tells other developers and Docker tooling which port the container intends to listen on at runtime. It does **NOT** actually publish or map ports.

```
EXPOSE 8080
```

- To actually publish the port, use -p in docker run: docker run -p 80:8080 myimage
- Used by docker-compose and orchestrators as hints for networking.

Note: Without -p or -P, EXPOSE does nothing to make the port accessible outside the container.

Q: 4. How do you handle multiple Python versions or base image versions in a Dockerfile build?

Multi-stage builds are the standard approach:

```
# Stage 1 — Python 3.11
FROM python:3.11-slim AS builder311
WORKDIR /app
COPY requirements_311.txt .
RUN pip install -r requirements_311.txt

# Stage 2 — Python 3.9
FROM python:3.9-slim AS builder39
WORKDIR /app
COPY requirements_39.txt .
RUN pip install -r requirements_39.txt
```

- **Build Args:** Use --build-arg to pass the base image version at build time: ARG PYTHON_VERSION=3.11 | FROM python:${PYTHON_VERSION}-slim
- **Separate Dockerfiles:** Maintain Dockerfile.py311 and Dockerfile.py39 and build with -f flag.
- **Docker Bake / Matrix builds:** Build multiple image variants in CI using a bake file or matrix strategy in GitHub Actions.

[#Linux] Linux & Scripting

Q: 1. What does chmod 777 do, and what is the meaning of permission code 5777?

chmod 777 grants read (4) + write (2) + execute (1) = 7 permissions to owner, group, and others. Every user on the system can read, write, and execute the file.

- **5777** — the leading digit is the special permissions bit:
- 5 = SUID (4) + Sticky Bit (1) | 7 = owner rwx | 7 = group rwx | 7 = others rwx
- **SUID (4):** File executes with the owner's privileges, not the caller's.
- **SGID (2):** File executes with the group's privileges; on directories, new files inherit the group.
- **Sticky Bit (1):** On directories (e.g., /tmp), only the file owner can delete their own files.

Note: chmod 777 is a security risk. Avoid in production. 5777 is extremely permissive — SUID + full permissions for everyone.

Q: 2. Which command substitutes a string or pattern within a particular directory in Linux?

sed (stream editor) is the standard tool for in-place substitution:

```
# Replace 'foo' with 'bar' in all .conf files in a directory
sed -i 's/foo/bar/g' /etc/myapp/*.conf

# Recursive replacement across a directory tree
find /etc/myapp -type f -name '*.conf' -exec sed -i 's/foo/bar/g' {} +
```

- The -i flag edits files in-place. Always test without -i first or use -i.bak for backups.

Q: 3. How do you filter or search for a specific word/pattern using grep or sed?

- **grep basics:**

```
grep 'error' app.log # Lines containing 'error'
grep -i 'error' app.log # Case-insensitive
grep -r 'TODO' ./src/ # Recursive search
grep -n 'error' app.log # Show line numbers
grep -v 'debug' app.log # Invert match (exclude)
grep -E 'error|warn' app.log # Extended regex (OR)
```

- **sed for filtering/extracting:**

```
sed -n '/error/p' app.log # Print only matching lines
sed 's/ERROR/CRITICAL/g' app.log # Replace in stream (no file edit)
sed -n '10,20p' app.log # Print lines 10-20
```

Q: 4. Write a Python script to search for a specific file within a folder or directory.

```
import os

def find_file(root_dir: str, target: str) -> list[str]:
"""Recursively search for target filename under root_dir."""
matches = []
for dirpath, dirnames, filenames in os.walk(root_dir):
if target in filenames:
matches.append(os.path.join(dirpath, target))
return matches

if __name__ == "__main__":
results = find_file("/home/user", "config.yaml")
if results:
for path in results:
print(f"Found: {path}")
else:
print("File not found.")
```

Uses os.walk for recursive traversal. Returns a list of all matching paths, not just the first match.

[#CI/CD #Git] CI/CD & Git

Q: 1. How do you build parallel stages in a Jenkins pipeline?

Use the **parallel** directive inside a stage (Declarative) or the parallel step (Scripted):

```
// Declarative Pipeline
stage('Test') {
parallel {
stage('Unit Tests') {
steps { sh 'make test-unit' }
}
stage('Integration Tests') {
steps { sh 'make test-integration' }
}
stage('Lint') {
steps { sh 'make lint' }
}
}
}
```

- Parallel stages run simultaneously on available agents, reducing total pipeline duration.
- Use failFast: true inside parallel to abort remaining branches if one fails.

Q: 2. What is the difference between git merge and git rebase?

git merge integrates changes by creating a new merge commit that ties together the histories of both branches. It preserves the complete history.

git rebase moves or replays commits from one branch onto another, rewriting commit history to appear linear.

Aspect	git merge	git rebase
History	Preserves all history + merge commit	Rewrites commits — linear history
Commit SHA	Originals unchanged	New SHAs created for rebased commits
Conflict handling	Resolved once at merge point	Resolved per commit being replayed
Use case	Feature branch -> main (shared repos)	Local cleanup before pushing
Safe for shared branches?	Yes	No — never rebase public branches

---

Q: 3. What is meant by 'staging' in Git?

The **staging area (index)** is an intermediate layer between your working directory and the Git repository. It lets you precisely control which changes go into a commit.

```
git add file.py # Stage specific file
git add -p # Interactively stage hunks
git status # See staged vs unstaged changes
git diff --staged # Review what is staged
git commit -m 'message' # Commit only staged changes
```

- Allows you to split a large set of changes into logical, atomic commits.
- Unstaged changes remain in the working directory and are not included in the next commit.

Q: 4. Explain the concept of Blue-Green Deployment.

Blue-Green deployment is a release strategy that maintains two identical production environments:

- **Blue:** The currently live environment serving all production traffic.
- **Green:** The new version environment, deployed and tested in parallel.
- **Cutover:** Traffic is switched from Blue to Green (via DNS, load balancer, or Ingress update) instantly — zero downtime.
- **Rollback:** If Green has issues, switch traffic back to Blue immediately.
- **Trade-off:** Requires double the infrastructure. Cost is offset by reliability.
- **Tools:** AWS CodeDeploy, Argo Rollouts, Kubernetes Service selector swap, ALB weighted target groups.

Q: 5. What is Git stash used for?

git stash temporarily shelves (stashes) uncommitted changes in the working directory so you can switch context without committing incomplete work.

```
git stash # Stash current changes
git stash pop # Reapply most recent stash + delete it
git stash apply # Reapply stash but keep it
git stash list # View all stashes
git stash drop # Delete a specific stash
```

- Use case: You're mid-feature and need to hotfix another branch urgently. Stash your work, fix the bug, then pop the stash to resume.

Q: 6. What is git commit used for, and are all commit SHAs the same after git rebase?

git commit records a snapshot of staged changes to the repository. Each commit stores: tree (file snapshot), author, timestamp, commit message, and parent commit reference.

After git rebase — No, commit SHAs change. Rebase replays each commit on a new base. Even if the content (diff) is identical, the parent commit changes, which changes the SHA hash. The old commits still exist temporarily in the reflog (git reflog) but are no longer part of the branch's history.

- This is why force-pushing after rebase (git push --force-with-lease) is required — the remote branch has the old SHAs.

Note: Never rebase commits that have already been pushed to a shared branch. It rewrites history and causes conflicts for other contributors.

End of Document · DevOps Interview Q&A Guide